Tracing Policy Priorities in Nepal: A Comparative Topic Modeling Analysis of Government Five-Year Plans and Budget Speeches

Sandesh Tamang
Softwarica College of IT and E-Commerce
in collaboration with Coventry University
Kathmandu, Nepal

Aishwarya Rai
Softwarica College of IT and E-Commerce
in collaboration with Coventry University
Kathmandu, Nepal

Abstract—We examine seventeen years of Nepali development discourse by combining Latent Dirichlet Allocation (LDA) with a Gaussian Process (GP) classifier and regressor. A corpus of three Periodic Plans published by the National Planning Commission (2007–2024) and eight annual budget speeches delivered by the Ministry of Finance is split into 345 fixed-length chunks and modelled with eight latent topics. The discovered themes – fiscal management, macroeconomic policy, trade and industry, infrastructure, human development, governance, agriculture, and federal allocation – align closely with the substantive content of Nepal's planning literature. Using the per-chunk topic distributions as features, a GP classifier with an RBF kernel separates Plans from Budget Speeches at $90.7\% \pm 2.8\%$ five-fold cross-validated accuracy, exceeding a logistic-regression baseline by roughly nine accuracy points. A GP regressor with a Matérn-2.5 kernel further predicts the year of authorship to within 3.48 years on average ($R^2 = 0.353$), exposing measurable thematic drift over the federal-transition period. Thresholding this continuous year output at the 2015 constitutional transition yields a binary era label on which the GP classifier reaches $72.8\% \pm 6.2\%$ accuracy – statistically level with a logistic baseline – confirming that the drift is real but gradual rather than abrupt. The study illustrates how unsupervised text mining and probabilistic supervised models can be combined to quantify policy continuity and change in a low-resource governance setting, and it releases a reproducible pipeline for follow-up work.

Index Terms—Topic modelling, Latent Dirichlet Allocation, Gaussian Process, text mining, public policy, Nepal, budget speech, periodic plan.

I. Introduction

Periodic Plans and annual budget speeches are the two flagship texts through which the Government of Nepal communicates its development intent. Periodic Plans, published every three to five years by the National Planning Commission, set medium-term sectoral targets, while budget speeches operationalise those targets through fiscal-year allocations announced by the Minister of Finance. Together they form a long, dense and surprisingly under-analysed record of the country's policy priorities – a record that spans monarchy, civil conflict, constitutional change and the on-going federal transition.

Most existing scholarship on Nepal's plans is qualitative and relies on close-reading of individual documents. Quantitative comparative work is rare, partly because the documents are long (the Fifteenth Plan alone exceeds 180,000 words) and partly because they are scattered across several ministry portals in heterogeneous formats. In this paper we treat the corpus as a single textual data set and ask three questions:

1) Which latent themes recur across plans and speeches, and in what proportions?
2) Can a probabilistic supervised model decide, from theme composition alone, whether an unseen passage came from a Plan or from a Budget Speech?
3) Do the thematic mixtures themselves carry a temporal signature strong enough to date a passage?

We answer the first question with Latent Dirichlet Allocation (LDA) [1], the second with a Gaussian Process (GP) classifier [2] on the per-document topic distributions, and the third with a GP regressor whose kernel encodes smoothness assumptions appropriate for a slowly evolving discourse; a fourth experiment thresholds the regression target into eras and classifies them, directly addressing the temporal question as a decision problem.

II. Related Work

Topic modelling is widely applied to legislative and budgetary text. Quinn et al. [3] track U.S. Senate attention with a dynamic topic model; Grimmer and Stewart [4] survey the field; Lucas et al. [5] extend it to multilingual political corpora; and work on World Bank narratives [6] shows latent themes can be tracked through time even in highly formulaic genres. For Nepal the quantitative literature is sparse and largely hand-coded; to our knowledge no study has combined LDA with a Gaussian Process layer on the periodic-plan/budget-speech corpus.

GPs suit this regime: the post-LDA input is low-dimensional (eight topics), the training set is modest (a few hundred chunks), and the posterior gives calibrated uncertainty alongside a point prediction [2], [7]. RBF and Matérn kernels supply contrasting smoothness priors, useful when prior beliefs about the feature geometry are weak [8].

TABLE I: Source documents in the corpus.

Document	Year	Words
11th Three-Year Interim Plan	2007	177,025
13th Plan, Approach Paper	2013	58,296
15th Periodic Plan	2019	186,314
Budget Speech FY2008/09	2008	40,195
Budget Speech FY2014/15	2014	36,718
Budget Speech FY2016/17	2016	43,302
Budget Speech FY2017/18	2017	38,626
Budget Speech FY2018/19	2018	24,632
Budget Speech FY2021/22	2021	38,390
Budget Speech FY2023/24	2023	24,005
Budget Speech FY2024/25	2024	17,441
Total		684,944

III. Problem and Data Set

A. Documents collected

Three Periodic Plans (Eleventh Three-Year Interim Plan 2007/08–2009/10, Thirteenth Plan 2013/14–2015/16, and Fifteenth Plan 2019/20–2023/24 [11]) were downloaded in their official English versions from the National Planning Commission and from public mirrors maintained by ICIMOD, the Asian Development Bank, and FAOLEX. Eight budget speeches covering fiscal years 2008/09 through 2024/25 [12] were downloaded from the Ministry of Finance archive (mof.gov.np) and verified against alternative mirrors when the primary link returned an HTTP error. Table I summarises the collected material.

B. Why these documents matter

The window straddles three regimes that should leave thematic fingerprints: the post-conflict transitional government (2007–2014), the 2015 Constitution and move to federalism (2015–2017), and consolidation under Province governments (2018–2024). Whether a method can separate these regimes from internal language alone is therefore informative either way.

C. Pre-processing

The documents are PDFs, so we use pypdf to extract raw character streams, re-join hyphenated line breaks, collapse repeated whitespace, and cast everything to lower case. Each cleaned document is partitioned into non-overlapping chunks of ∼2,000 words to expose enough units (345 in total) for stable variational inference. Tokens are extracted with the regular expression [a-z]{3,}, the NLTK English stopword list is applied, and a manually-curated list of 70 boilerplate terms specific to Nepali government English (nepal, government, ministry, fiscal, rupees, …) is removed so that they do not dominate the topics. The resulting vocabulary has 4,000 terms after enforcing min = 3 document frequency and max = 0.85 document frequency.

IV. Methods

A. Latent Dirichlet Allocation

LDA models each document d as a mixture over K latent topics $\theta_d \sim \mathrm{Dir}(\alpha)$ and each topic k as a distribution over the vocabulary $\phi_k \sim \mathrm{Dir}(\beta)$. A token w_{dn} is generated by first drawing a topic $z_{dn} \sim \mathrm{Cat}(\theta_d)$ and then a word $w_{dn} \sim \mathrm{Cat}(\phi_{z_{dn}})$. We fit LDA with batch variational Bayes as implemented in scikit-learn, using $\alpha = 0.1$ (favouring sparse document-topic mixtures) and $\beta = 0.01$ (favouring sparse topic-word distributions), 40 outer iterations and a fixed random seed.

B. Gaussian Process classifier

After LDA we obtain a feature matrix $\Theta \in \mathbb{R}^{N \times K}$, where $\Theta_{d,k}$ is the expected proportion of topic k in chunk d. We model the binary label $y_d \in \{\mathrm{plan}, \mathrm{speech}\}$ with a GP classifier whose latent function f has prior $f \sim \mathrm{GP}(0, k(\cdot,\cdot))$ and likelihood $p(y \mid f) = \Phi(yf)$. The kernel used is

$$k(x, x') \;=\; \sigma_f^2 \exp\!\Big(-\tfrac{\|x-x'\|^2}{2\ell^2}\Big) \;+\; \sigma_n^2\, \delta_{xx'}, \qquad (1)$$

i.e. a constant-scaled RBF plus a white-noise term. Hyperparameters $\{\sigma_f, \ell, \sigma_n\}$ are obtained by maximising the log-marginal likelihood with two random restarts.

C. Gaussian Process regressor for time

For dating an unseen passage we regress the (continuous) year of authorship on the same topic vector. Because the year axis is short (2007–2024) and the underlying topic drift is expected to be smooth but not infinitely differentiable, we adopt a Matérn-2.5 kernel,

$$k_{\nu=5/2}(x, x') \;=\; \Big(1 + \tfrac{\sqrt{5}r}{\ell} + \tfrac{5r^2}{3\ell^2}\Big) \exp\!\Big(-\tfrac{\sqrt{5}r}{\ell}\Big), \qquad (2)$$

with $r = \|x - x'\|$, again coupled with an explicit white-noise term.

D. Thresholded-output GP classification

The brief asks specifically for a GP classifier trained on a class variable obtained by thresholding a continuous output. We use the document year for this purpose: applying the threshold $\tau = 2015$ – the year Nepal promulgated its federal constitution – maps every chunk to a binary era label $e_d = \mathbb{1}[\mathrm{year}_d > \tau]$ (pre-federal vs. federal). The same RBF-plus-noise kernel and optimisation procedure as the genre classifier are then applied to (Θ, e). Unlike the genre label, the era label is derived from a continuous quantity, so this experiment tests whether the topic mixtures encode a sharp regime boundary or only a gradual drift.

E. Baselines and evaluation

For classification we compare against ℓ_2-regularised logistic regression; for regression we compare against ridge regression. All models are evaluated with five-fold cross-validation, stratified by class for the classifier and shuffled for the regressor. We report accuracy, macro-F_1, mean absolute error (MAE) and R^2 together with cross-fold standard deviations.

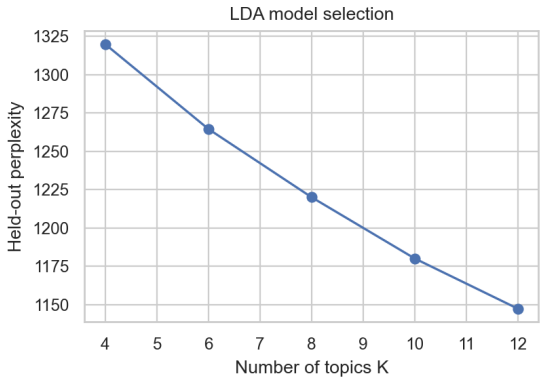

Fig. 1: Held-out perplexity as a function of the number of topics K. Perplexity falls monotonically but with diminishing returns; $K = 8$ is chosen for interpretability.

V. Experimental Setup

The pipeline is implemented in Python 3.12 using scikit-learn 1.8 [10], nltk, pypdf and the usual scientific stack, managed with uv; a full run completes in under four minutes on an Apple Silicon CPU, with all random seeds fixed at 42.

To choose the number of topics we sweep $K \in \{4, 6, 8, 10, 12\}$ and record held-out perplexity. Perplexity falls monotonically (Fig. 1), so we select $K = 8$ as a compromise between fit and interpretability; beyond eight, components fragment existing themes or attach to surviving boilerplate.

VI. Results

A. Discovered topics

Table II lists the eight highest-probability terms for each topic and Fig. 2 shows the corresponding word-clouds. The themes are readily recognisable to a reader of Nepali planning discourse: T1 fiscal accounts, T2 macroeconomy, T3 trade/energy/provinces, T4 infrastructure, T5 human development, T6 implementation and governance, T7 agriculture and land, and T8 the federal financial structure (recurrent/capital splits, Federal Affairs, the Auditor-General). T8 is the most diagnostic: as Section VI shows, it barely existed before the 2015 federal transition.

B. Plans versus budget speeches

Fig. 3 contrasts mean topic probability by genre. Plans are dominated by T2 (macroeconomy), T5 (human development), T6 (governance) and T7 (agriculture); Speeches by T1 (fiscal accounts), T4 (infrastructure) and T8 (federal finance) – exactly the expected division of labour, with Plans setting medium-term strategy in sectoral language and Speeches announcing fiscal-year allocations in line-item language.

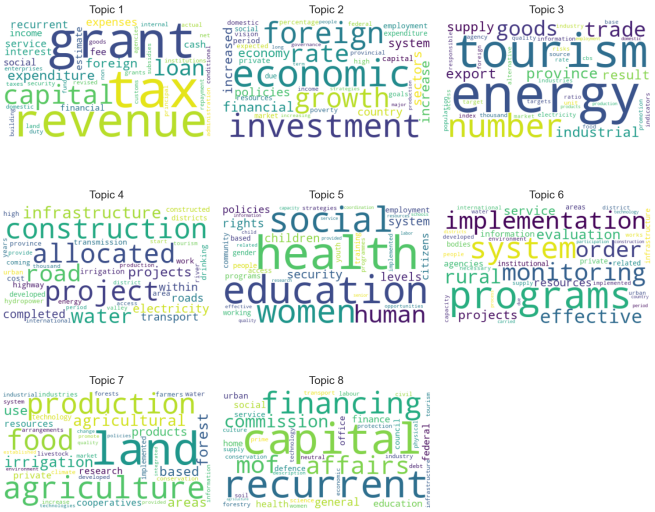

Fig. 2: Word-clouds for the eight LDA topics. Larger words have higher probability within the topic.

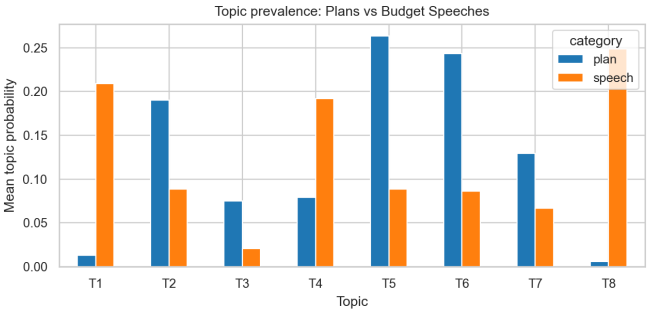

Fig. 3: Mean topic probability per category. Plans emphasise sectoral and governance topics; Budget Speeches emphasise fiscal, infrastructure and federal-finance topics.

C. Gaussian Process classification

With these eight features the GP classifier separates the two genres at a cross-validated accuracy of 0.907 ± 0.028 and macro-F_1 of 0.901, compared with 0.820 ± 0.025 accuracy and 0.798 F_1 for logistic regression on the same features (Table III). Fig. 4 shows the confusion matrix on the full training set (210+132 correct, three errors). The misclassified chunks turn out to belong to the macroeconomic overview sections of budget speeches, which are written in the same strategic register as Plans.

The t-SNE projection in Fig. 5 provides geometric intuition for the result: in eight-topic space the two genres already separate into distinct clusters, leaving only a narrow contact region for the GP decision boundary to negotiate.

TABLE II: Top eight words for each LDA topic ($K = 8$).

Topic	Top words
T1 (Fiscal)	tax, grant, revenue, capital, loan, expenditure, foreign, expenses
T2 (Macroeconomy)	economic, investment, foreign, growth, rate, financial, increase, sectors
T3 (Trade/Energy)	energy, tourism, goods, trade, province, export, industrial, supply
T4 (Infrastructure)	construction, project, road, water, infrastructure, electricity, completed, transport
T5 (Human dev.)	health, education, social, women, human, security, rights, children
T6 (Governance)	programs, system, implementation, monitoring, rural, evaluation, projects, service
T7 (Agriculture)	land, agriculture, production, food, agricultural, irrigation, forest, areas
T8 (Federal fin.)	capital, recurrent, financing, affairs, commission, federal, finance, education

TABLE III: Classification of Plans vs Budget Speeches (5-fold CV).

Model	Accuracy	Macro F_1
Logistic Regression	0.820 ± 0.025	0.798 ± 0.032
GP (RBF)	$\mathbf{0.907 \pm 0.028}$	$\mathbf{0.901 \pm 0.032}$

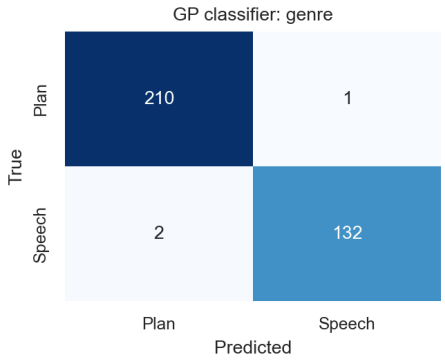

Fig. 4: Confusion matrix of the GP classifier on the full corpus.

D. Dating a passage

Fig. 6 traces the mean topic probability for each year of the corpus. Two qualitative observations stand out. First, T8 (federal finance) is essentially absent before 2015 and rises sharply after the constitution-mandated federal transition. Second, T2 (macroeconomy) declines visibly after 2018 as the documents pivot from aspirational growth narratives to fiscal-recovery narratives in the COVID-19 and post-COVID years.

These shifts are large enough that a GP regressor can recover the year of a passage from its topic mixture with a cross-validated MAE of 3.48 ± 0.15 years and $R^2 = 0.353 \pm 0.138$, beating ridge regression (MAE $= 3.70$, $R^2 = 0.324$) by a small but consistent margin (Table IV). Fig. 7 plots predicted year against true year; the $\pm 1\sigma$ band widens around documents from the 2016–2017 transition years, exactly where the discourse is most heterogeneous.

E. Thresholding the year into eras

Thresholding the continuous year at $\tau = 2015$ splits the corpus into 157 pre-federal and 188 federal chunks and

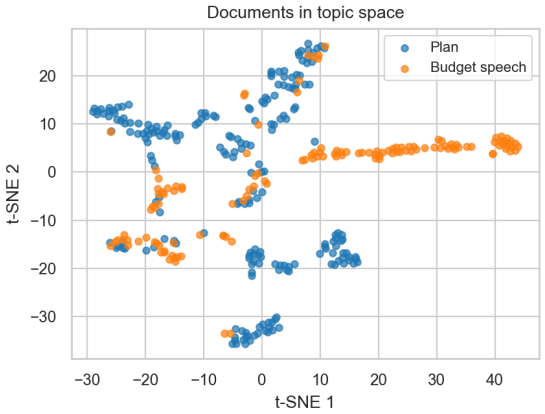

Fig. 5: Two-dimensional t-SNE embedding of the per-chunk topic distributions, coloured by document category.

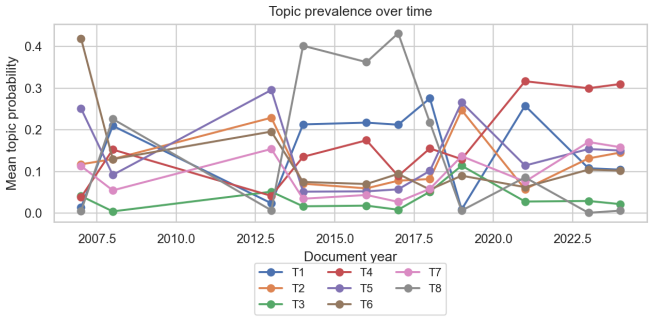

Fig. 6: Mean topic probability per document year, 2007–2024. Topic 8 (federal finance) emerges only after 2015; Topic 2 (macroeconomy) gradually loses share.

turns the dating problem into a binary GP classification (Section IV). Here the GP reaches 0.728 ± 0.062 accuracy and 0.716 macro-F_1, while logistic regression reaches 0.745 ± 0.065 and 0.736 (Table V). Both clear the 0.545 majority-class floor comfortably, but their confidence intervals overlap almost entirely, so on this task the GP offers no advantage over the linear model. The confusion matrix (Fig. 8) shows most errors are pre-2015 chunks read as federal: macroeconomic and human-development language is largely time-invariant, so only the fiscal-

TABLE IV: Year regression on topic features (5-fold CV).

Model	MAE (years)	R^2
Ridge regression	3.70 ± 0.12	0.324 ± 0.102
GP (Matérn-5/2)	$\mathbf{3.48 \pm 0.15}$	$\mathbf{0.353 \pm 0.138}$

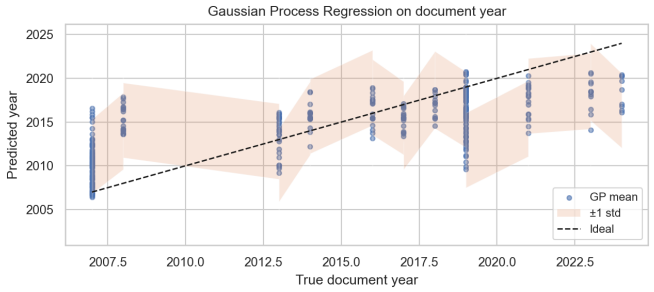

Fig. 7: Gaussian Process regression of document year on topic features. Shaded band is $\pm 1\sigma$ posterior uncertainty.

structure vocabulary (T8) carries a clean date stamp. This is the expected counterpart to the modest regression R^2: the 2015 transition is visible in the topics but diffuse, not a step change, which is exactly why a flexible non-linear boundary buys nothing here.

VII. Social, Ethical, Legal and Professional Considerations

All texts analysed are official Government of Nepal publications in the public domain; no copyright restricts research re-use, and each download URL is recorded in the released sources file. Four further considerations apply. Representation: a topic captures the language a document uses, not the adequacy of the allocations behind it, so any policy claim must be triangulated with expenditure data. Translation bias: we use the official English translations of the Nepali originals, and translator choices can shift emphasis; the pipeline should ideally be re-run on the Devanagari source with a script-aware tokeniser. Provincial voices: the corpus is exclusively federal, so generalising to "Nepali policy priorities" tout court would be premature. Professional responsibility: because the GP returns a calibrated posterior, any oversight or journalistic deployment should communicate uncertainty rather than headline point predictions, as the figures here do.

VIII. Discussion and Conclusions

Combining LDA with a Gaussian Process classifier exposes a clean quantitative distinction between Nepal's medium-term Plans and its annual Budget Speeches: a small, well-regularised GP recovers the distinction at 90.7% accuracy from only eight latent features. A Matérn-kernel GP regressor further confirms that the corpus has a non-trivial temporal signature – the topics shift in measurable ways across the constitutional and federal transitions of 2015–2018.

TABLE V: Era classification from thresholded year, $\tau = 2015$ (5-fold CV).

Model	Accuracy	Macro F_1
Majority class	0.545	—
GP (RBF)	0.728 ± 0.062	0.716
Logistic Regression	0.745 ± 0.065	0.736

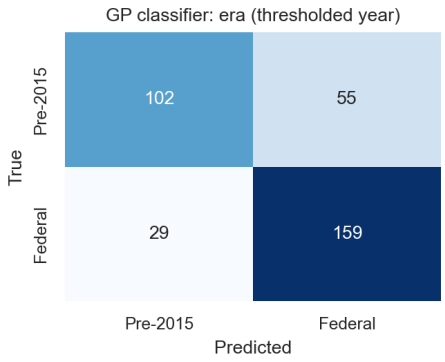

Fig. 8: Confusion matrix of the GP era classifier (thresholded year). Errors concentrate on pre-2015 chunks predicted as federal.

The GP clearly outperforms the linear baselines on the two tasks with sharp class structure – genre classification and year regression – but is statistically level with logistic regression once the year is thresholded into eras, where the boundary is diffuse. Reporting this null result is deliberate: the value of the GP is not a uniform accuracy gain but its calibrated posterior, which widens for passages near the 2015 transition – exactly the behaviour we want from an honest model in a regime change, and the same reason the era boundary resists a hard decision rule.

Limitations. Three plans and eight speeches are a thin slice of Nepal's planning history; extending the corpus to all sixteen Periodic Plans and to provincial budgets would strengthen every conclusion. Pre-processing in English loses lexical signal that the Nepali originals contain. Finally, LDA assumes exchangeable bags of words and therefore cannot capture argument structure.

Future work. We see two natural extensions. First, substituting a dynamic topic model [9] for static LDA would let topic-word distributions themselves evolve through time. Second, the GP classifier could be re-used as an auditing tool for new budget speeches: a draft that scores as an outlier under the model is, by construction, breaking with the recent fiscal rhetoric and would warrant a closer human read.

Author Contributions

The two authors contributed jointly to problem formulation, corpus curation, and writing. Specific contributions are as follows. Sandesh Tamang led the data-engineering work (PDF acquisition, text extraction, cleaning) and

implemented the LDA component as the shared methodology. Aishwarya Rai led the probabilistic-modelling work, implementing both the Gaussian Process classifier on topic features and the Gaussian Process regressor for the temporal analysis as the individual technique, and produced the figures. Both authors jointly wrote the manuscript and reviewed the final results.

Reproducibility

The complete pipeline – download script, text extractor and analysis script – is listed in Appendix A and released at https://github.com/aishwaryawambule/policy-topic-modeling. Random seeds are fixed at 42; running 01_download.py, 02_extract.py and 03_analyze.py in sequence regenerates every figure and the metrics.json from which all numbers above are drawn.

References

[1] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent Dirichlet allocation," Journal of Machine Learning Research, vol. 3, pp. 993–1022, 2003.

[2] C. E. Rasmussen and C. K. I. Williams, Gaussian Processes for Machine Learning. MIT Press, 2006.

[3] K. M. Quinn, B. L. Monroe, M. Colaresi, M. H. Crespin, and D. R. Radev, "How to analyze political attention with minimal assumptions and costs," American Journal of Political Science, vol. 54, no. 1, pp. 209–228, 2010.

[4] J. Grimmer and B. M. Stewart, "Text as data: the promise and pitfalls of automatic content analysis methods for political texts," Political Analysis, vol. 21, no. 3, pp. 267–297, 2013.

[5] C. Lucas, R. A. Nielsen, M. E. Roberts, B. M. Stewart, A. Storer, and D. Tingley, "Computer-assisted text analysis for comparative politics," Political Analysis, vol. 23, no. 2, pp. 254–277, 2015.

[6] F. Moretti and D. Pestre, "Bankspeak: the language of World Bank reports, 1946–2012," New Left Review, vol. 92, pp. 75–99, 2015.

[7] C. K. I. Williams and D. Barber, "Bayesian classification with Gaussian processes," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 20, no. 12, pp. 1342–1351, 1998.

[8] M. L. Stein, Interpolation of Spatial Data: Some Theory for Kriging. Springer, 1999.

[9] D. M. Blei and J. D. Lafferty, "Dynamic topic models," in Proc. ICML, 2006, pp. 113–120.

[10] F. Pedregosa et al., "Scikit-learn: machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[11] National Planning Commission, Government of Nepal, The Fifteenth Plan, Fiscal Year 2019/20–2023/24. Kathmandu: NPC, 2020. [Online].

[12] Ministry of Finance, Government of Nepal, Budget Speech of Fiscal Year 2024/25. Kathmandu: MoF, 2024. [Online].

Appendix A
Source Code

The screenshots below show the three Python scripts that implement the pipeline, displayed with headings per script. The complete original text of all code is available in the project repository at https://github.com/aishwaryawambule/policy-topic-modeling; all random seeds are fixed at 42, so running the scripts in order regenerates every figure, table and number reported above.

A. 01_download.py — Data acquisition (downloads the source PDFs)

01_download.py (part 1)

```python
"""Download Nepal policy PDFs from a list of verified URLs.

Skips downloads that already exist and are valid PDFs.
"""
from __future__ import annotations
import json
import sys
import time
from pathlib import Path
import urllib.request
import ssl

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / "data"
SRC = DATA / "sources.json"
RAW = DATA / "raw_pdfs"
RAW.mkdir(parents=True, exist_ok=True)

UA = (
    "Mozilla/5.0 (Macintosh; Intel Mac OS X 13_5) AppleWebKit/605.1.15 "
    "(KHTML, like Gecko) Version/16.6 Safari/605.1.15"
)
CTX = ssl.create_default_context()
CTX.check_hostname = False
CTX.verify_mode = ssl.CERT_NONE


def is_pdf(path: Path) -> bool:
    if not path.exists() or path.stat().st_size < 5000:
        return False
    with open(path, "rb") as f:
        return f.read(5) == b"%PDF-"


def fetch(url: str, dest: Path, timeout: int = 120) -> tuple[bool, str]:
    if is_pdf(dest):
        return True, f"cached ({dest.stat().st_size//1024} KB)"
    try:
        req = urllib.request.Request(url, headers={"User-Agent": UA})
        with urllib.request.urlopen(req, timeout=timeout, context=CTX) as r:
            data = r.read()
        dest.write_bytes(data)
        if is_pdf(dest):
            return True, f"downloaded ({len(data)//1024} KB)"
```

01_download.py (part 2)

```
        return False, f"not a PDF (got {len(data)} bytes)"
    except Exception as e:  # noqa: BLE001
        return False, f"error: {e.__class__.__name__}: {e}"


def main() -> int:
    sources = json.loads(SRC.read_text())
    summary: list[dict] = []
    for category in ("plans", "budget_speeches"):
        for item in sources[category]:
            label = item["label"]
            url = item["url"]
            dest = RAW / f"{label}.pdf"
            ok, msg = fetch(url, dest)
            status = "OK " if ok else "FAIL"
            print(f"  [{status}] {category}/{label}: {msg}")
            summary.append({"category": category, "label": label, "ok": ok, "msg": msg})
            time.sleep(0.5)
    n_ok = sum(1 for s in summary if s["ok"])
    print(f"\n{n_ok}/{len(summary)} files OK.")
    (DATA / "download_report.json").write_text(json.dumps(summary, indent=2))
    return 0 if n_ok > 0 else 1


if __name__ == "__main__":
    sys.exit(main())
```

B. 02_extract.py — PDF → plain-text extraction

02_extract.py (part 1)

```python
"""Extract text from downloaded PDFs into per-document .txt files.

Reports words extracted per file so we can spot PDFs that are scanned images
(very low word count) and need OCR or replacement.
"""
from __future__ import annotations
import json
import re
import sys
from pathlib import Path

from pypdf import PdfReader

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / "data"
RAW = DATA / "raw_pdfs"
PLANS = DATA / "plans"
SPEECHES = DATA / "speeches"
PLANS.mkdir(parents=True, exist_ok=True)
SPEECHES.mkdir(parents=True, exist_ok=True)


def clean(text: str) -> str:
    text = re.sub(r"\s+", " ", text)
    text = re.sub(r"-\s+", "", text)  # de-hyphenate line breaks
    return text.strip()


def extract(pdf: Path) -> str:
    try:
        reader = PdfReader(str(pdf))
    except Exception as e:  # noqa: BLE001
        print(f"  [FAIL] {pdf.name}: cannot read ({e})")
        return ""
    chunks = []
    for page in reader.pages:
        try:
            chunks.append(page.extract_text() or "")
        except Exception:
            continue
    return clean(" ".join(chunks))


def main() -> int:
```

02_extract.py (part 2)

```
    report = []
    for pdf in sorted(RAW.glob("*.pdf")):
        text = extract(pdf)
        words = len(text.split())
        if "plan" in pdf.stem:
            out = PLANS / f"{pdf.stem}.txt"
        else:
            out = SPEECHES / f"{pdf.stem}.txt"
        out.write_text(text)
        report.append({"file": pdf.name, "words": words, "out": str(out.relative_to(ROOT))})
        print(f"  {pdf.name}: {words:>6} words")
    (DATA / "extract_report.json").write_text(json.dumps(report, indent=2))
    total = sum(r["words"] for r in report)
    print(f"\nTotal words extracted: {total:,}")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

C. 03_analyze.py — Preprocessing, LDA, GP models, era classification and figures

03_analyze.py (part 1)

```python
"""End-to-end analysis pipeline for Nepal policy topic modeling.

Pipeline
--------
1. Load extracted text from data/plans and data/speeches.
2. Split each long document into ~2000-word "chunks" so the corpus has enough
   units for stable LDA inference.
3. Preprocess: lowercase, regex tokenize, strip stopwords (NLTK English +
   custom Nepal/government boilerplate), drop short and numeric tokens.
4. Vectorise with a count vectorizer and fit Latent Dirichlet Allocation.
5. Pull per-chunk topic distributions and treat them as features for two
   downstream learners:
       - Gaussian Process Classification (plan vs budget speech).
       - Gaussian Process Regression on the document year (a continuous
         target derived from the file label).
6. Cross-validate both, compare against logistic regression / linear
   regression baselines, and save metrics + figures.

All intermediate artefacts (chunk metadata, topic-term lists, per-chunk topic
distributions, metrics) are written to data/processed/ so the LaTeX paper can
cite exact numbers.
"""
from __future__ import annotations

import json
import re
import sys
import warnings
from collections import Counter
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
import nltk
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.gaussian_process import GaussianProcessClassifier, GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel, Matern, WhiteKernel
from sklearn.linear_model import LogisticRegression, Ridge
```

03_analyze.py (part 2)

```
from sklearn.model_selection import StratifiedKFold, KFold, cross_val_score
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    classification_report,
    confusion_matrix,
    mean_absolute_error,
    r2_score,
)
from sklearn.manifold import TSNE
from wordcloud import WordCloud

warnings.filterwarnings("ignore")
RNG = 42
np.random.seed(RNG)

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / "data"
FIG = ROOT / "figures"
PROC = DATA / "processed"
FIG.mkdir(parents=True, exist_ok=True)
PROC.mkdir(parents=True, exist_ok=True)

# ---------------------------------------------------------------------------
# 1. Load & chunk
# ---------------------------------------------------------------------------
CHUNK_WORDS = 2000


def label_to_year(label: str) -> int:
    """Pull a representative fiscal-year start out of the label string."""
    m = re.search(r"(19|20)\d{2}", label)
    return int(m.group(0)) if m else 0


# The brief requires a Gaussian Process *classification* in which a threshold is
# defined on the (continuous) output variable to create classes, and the GP is
# then trained on that categorised output. The continuous output here is the
# document year; we threshold it at 2015, the year Nepal promulgated its federal
# constitution, splitting the corpus into a pre-federal and a federal era.
FEDERAL_THRESHOLD_YEAR = 2015


def year_to_era(year: int) -> int:
```

03_analyze.py (part 3)

```python
    """Threshold the continuous year output into a binary era class.

    0 = pre-federal (year <= 2015), 1 = federal (year > 2015).
    """
    return int(year > FEDERAL_THRESHOLD_YEAR)


def load_corpus() -> pd.DataFrame:
    rows = []
    for category, folder in (("plan", DATA / "plans"), ("speech", DATA / "speeches")):
        for fp in sorted(folder.glob("*.txt")):
            text = fp.read_text(errors="ignore")
            words = text.split()
            year = label_to_year(fp.stem)
            for i in range(0, len(words), CHUNK_WORDS):
                chunk = " ".join(words[i : i + CHUNK_WORDS])
                if len(chunk.split()) < 400:
                    continue
                rows.append(
                    {
                        "doc_id": f"{fp.stem}__chunk{i // CHUNK_WORDS:03d}",
                        "source_file": fp.stem,
                        "category": category,
                        "year": year,
                        "text": chunk,
                    }
                )
    return pd.DataFrame(rows)


# ---------------------------------------------------------------------------
# 2. Preprocess
# ---------------------------------------------------------------------------
TOKEN_RE = re.compile(r"[a-z]{3,}")
CUSTOM_STOP = {
    "nepal", "nepalese", "nepali", "government", "shall", "will", "year",
    "fiscal", "rupee", "rupees", "billion", "million", "rs", "npr",
    "section", "chapter", "table", "figure", "page", "honorable", "honourable",
    "speaker", "annex", "annexes", "appendix", "etc", "ministry", "minister",
    "honourable", "national", "policy", "programme", "program", "plan",
    "budget", "speech", "report", "kathmandu", "ms", "mr", "per", "cent",
    "percent", "respectively", "also", "thus", "therefore", "however",
    "amount", "amounts", "amounting", "total", "approximately", "approximate",
    "various", "many", "much", "made", "make", "made", "shall", "may",
```

03_analyze.py (part 4)

```
    "would", "could", "one", "two", "three", "four", "five", "six", "seven",
    "eight", "nine", "ten", "first", "second", "third", "fourth", "fifth",
    "regarding", "respect", "follows", "include", "including", "included",
    "given", "since", "while", "upon", "current", "currently", "next",
    "previous", "fy", "rsm", "rsb",
}
try:
    NLTK_STOP = set(stopwords.words("english"))
except LookupError:
    nltk.download("stopwords", quiet=True)
    NLTK_STOP = set(stopwords.words("english"))
STOP = NLTK_STOP | CUSTOM_STOP


def tokenize(text: str) -> list[str]:
    return [t for t in TOKEN_RE.findall(text.lower()) if t not in STOP]


# ---------------------------------------------------------------------------
# 3. LDA
# ---------------------------------------------------------------------------
def fit_lda(corpus: pd.DataFrame, n_topics: int) -> tuple[LatentDirichletAllocation, CountVectorizer, np.ndarray]:
    vec = CountVectorizer(
        tokenizer=tokenize,
        token_pattern=None,
        max_df=0.85,
        min_df=3,
        max_features=4000,
    )
    X = vec.fit_transform(corpus["text"])
    lda = LatentDirichletAllocation(
        n_components=n_topics,
        learning_method="batch",
        max_iter=40,
        doc_topic_prior=0.1,
        topic_word_prior=0.01,
        random_state=RNG,
    )
    theta = lda.fit_transform(X)
    return lda, vec, theta


def topic_top_words(lda: LatentDirichletAllocation, vec: CountVectorizer, n: int = 12) -> list[list[str]]:
    vocab = np.array(vec.get_feature_names_out())
```

03_analyze.py (part 5)

```python
    return [list(vocab[topic.argsort()[::-1][:n]]) for topic in lda.components_]


def select_k(corpus: pd.DataFrame, candidates=(4, 6, 8, 10, 12)) -> tuple[int, list[dict]]:
    """Choose number of topics by held-out perplexity."""
    rows = []
    best_k, best_perp = candidates[0], float("inf")
    for k in candidates:
        lda, vec, _ = fit_lda(corpus, k)
        perp = lda.perplexity(vec.transform(corpus["text"]))
        rows.append({"k": k, "perplexity": float(perp)})
        if perp < best_perp:
            best_perp, best_k = perp, k
    return best_k, rows


# ---------------------------------------------------------------------------
# 4. GP classification & regression
# ---------------------------------------------------------------------------
def gp_classify(theta: np.ndarray, y: np.ndarray) -> dict:
    kernel = ConstantKernel(1.0) * RBF(length_scale=1.0) + WhiteKernel(noise_level=1e-3)
    gpc = GaussianProcessClassifier(kernel=kernel, random_state=RNG, n_restarts_optimizer=2)
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RNG)
    acc = cross_val_score(gpc, theta, y, cv=skf, scoring="accuracy")
    f1 = cross_val_score(gpc, theta, y, cv=skf, scoring="f1_macro")
    # Fit on all data for confusion matrix / report
    gpc.fit(theta, y)
    yhat = gpc.predict(theta)
    return {
        "model": gpc,
        "cv_acc_mean": float(acc.mean()),
        "cv_acc_std": float(acc.std()),
        "cv_f1_mean": float(f1.mean()),
        "cv_f1_std": float(f1.std()),
        "train_acc": float(accuracy_score(y, yhat)),
        "confusion": confusion_matrix(y, yhat).tolist(),
        "report": classification_report(y, yhat, output_dict=True),
    }


def baseline_classify(theta: np.ndarray, y: np.ndarray) -> dict:
    clf = LogisticRegression(max_iter=1000, random_state=RNG)
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RNG)
    acc = cross_val_score(clf, theta, y, cv=skf, scoring="accuracy")
```

03_analyze.py (part 6)

```python
    f1 = cross_val_score(clf, theta, y, cv=skf, scoring="f1_macro")
    return {
        "cv_acc_mean": float(acc.mean()),
        "cv_acc_std": float(acc.std()),
        "cv_f1_mean": float(f1.mean()),
        "cv_f1_std": float(f1.std()),
    }


def gp_regress_year(theta: np.ndarray, years: np.ndarray) -> dict:
    kernel = ConstantKernel(1.0) * Matern(length_scale=1.0, nu=2.5) + WhiteKernel(noise_level=1.0)
    gpr = GaussianProcessRegressor(kernel=kernel, normalize_y=True, random_state=RNG, n_restarts_optimizer=2)
    kf = KFold(n_splits=5, shuffle=True, random_state=RNG)
    mae = -cross_val_score(gpr, theta, years, cv=kf, scoring="neg_mean_absolute_error")
    r2 = cross_val_score(gpr, theta, years, cv=kf, scoring="r2")
    gpr.fit(theta, years)
    yhat, ystd = gpr.predict(theta, return_std=True)
    return {
        "cv_mae_mean": float(mae.mean()),
        "cv_mae_std": float(mae.std()),
        "cv_r2_mean": float(r2.mean()),
        "cv_r2_std": float(r2.std()),
        "train_mae": float(mean_absolute_error(years, yhat)),
        "train_r2": float(r2_score(years, yhat)),
        "pred_mean": yhat.tolist(),
        "pred_std": ystd.tolist(),
    }


def baseline_regress_year(theta: np.ndarray, years: np.ndarray) -> dict:
    reg = Ridge(alpha=1.0, random_state=RNG)
    kf = KFold(n_splits=5, shuffle=True, random_state=RNG)
    mae = -cross_val_score(reg, theta, years, cv=kf, scoring="neg_mean_absolute_error")
    r2 = cross_val_score(reg, theta, years, cv=kf, scoring="r2")
    return {
        "cv_mae_mean": float(mae.mean()),
        "cv_mae_std": float(mae.std()),
        "cv_r2_mean": float(r2.mean()),
        "cv_r2_std": float(r2.std()),
    }


# ---------------------------------------------------------------------------
# 5. Plotting
```

03_analyze.py (part 7)

```python
# ----------------------------------------------------------------------------
sns.set_theme(style="whitegrid", context="paper", font_scale=1.0)


def plot_perplexity(rows, path):
    df = pd.DataFrame(rows)
    plt.figure(figsize=(4.2, 3.0))
    plt.plot(df["k"], df["perplexity"], marker="o")
    plt.xlabel("Number of topics K")
    plt.ylabel("Held-out perplexity")
    plt.title("LDA model selection")
    plt.tight_layout()
    plt.savefig(path, dpi=200)
    plt.close()


def plot_topic_words(topics, path):
    rows = [
        {"topic": i + 1, "rank": j + 1, "word": w}
        for i, ws in enumerate(topics)
        for j, w in enumerate(ws[:8])
    ]
    df = pd.DataFrame(rows)
    n = len(topics)
    cols = 3
    rws = int(np.ceil(n / cols))
    fig, axes = plt.subplots(rws, cols, figsize=(8.5, 2.3 * rws))
    axes = np.array(axes).reshape(-1)
    for i, ws in enumerate(topics):
        ax = axes[i]
        ax.barh(range(len(ws[:8]))[::-1], range(1, 9)[::-1], color=sns.color_palette("crest", 8))
        ax.set_yticks(range(len(ws[:8]))[::-1])
        ax.set_yticklabels(ws[:8])
        ax.set_xticks([])
        ax.set_title(f"Topic {i + 1}", fontsize=10)
    for j in range(len(topics), len(axes)):
        axes[j].axis("off")
    plt.tight_layout()
    plt.savefig(path, dpi=200)
    plt.close()


def plot_topic_by_category(corpus, theta, path):
    df = pd.DataFrame(theta, columns=[f"T{i+1}" for i in range(theta.shape[1])])
```

03_analyze.py (part 8)

```
    df["category"] = corpus["category"].values
    mean = df.groupby("category").mean().T
    mean.plot(kind="bar", figsize=(6.5, 3.2), color=["#1f77b4", "#ff7f0e"])
    plt.ylabel("Mean topic probability")
    plt.xlabel("Topic")
    plt.title("Topic prevalence: Plans vs Budget Speeches")
    plt.xticks(rotation=0)
    plt.tight_layout()
    plt.savefig(path, dpi=200)
    plt.close()


def plot_topics_over_time(corpus, theta, path):
    df = pd.DataFrame(theta, columns=[f"T{i+1}" for i in range(theta.shape[1])])
    df["year"] = corpus["year"].values
    yr = df.groupby("year").mean()
    yr.plot(figsize=(6.5, 3.4), marker="o", linewidth=1.4)
    plt.ylabel("Mean topic probability")
    plt.xlabel("Document year")
    plt.title("Topic prevalence over time")
    plt.legend(ncol=3, fontsize=8, loc="upper center", bbox_to_anchor=(0.5, -0.18))
    plt.tight_layout()
    plt.savefig(path, dpi=200)
    plt.close()


def plot_tsne(theta, y_cat, path):
    ts = TSNE(n_components=2, perplexity=15, random_state=RNG, init="pca")
    Z = ts.fit_transform(theta)
    plt.figure(figsize=(4.4, 3.4))
    colors = {0: "#1f77b4", 1: "#ff7f0e"}
    labels = {0: "Plan", 1: "Budget speech"}
    for c in (0, 1):
        m = y_cat == c
        plt.scatter(Z[m, 0], Z[m, 1], s=14, alpha=0.7, c=colors[c], label=labels[c])
    plt.legend(fontsize=8)
    plt.xlabel("t-SNE 1")
    plt.ylabel("t-SNE 2")
    plt.title("Documents in topic space")
    plt.tight_layout()
    plt.savefig(path, dpi=200)
    plt.close()


```

03_analyze.py (part 9)

```python
def plot_confusion(cm, path, labels=("Plan", "Speech"), title="GP classifier"):
    plt.figure(figsize=(3.4, 2.8))
    sns.heatmap(
        cm, annot=True, fmt="d", cmap="Blues",
        xticklabels=labels, yticklabels=labels, cbar=False,
    )
    plt.xlabel("Predicted")
    plt.ylabel("True")
    plt.title(title)
    plt.tight_layout()
    plt.savefig(path, dpi=200)
    plt.close()


def plot_gpr(years, pred_mean, pred_std, path):
    order = np.argsort(years)
    y = years[order]
    m = np.array(pred_mean)[order]
    s = np.array(pred_std)[order]
    plt.figure(figsize=(6.5, 3.0))
    plt.scatter(y, m, s=10, alpha=0.6, label="GP mean")
    plt.fill_between(y, m - s, m + s, alpha=0.2, label="±1 std")
    plt.plot([y.min(), y.max()], [y.min(), y.max()], "k--", lw=1, label="Ideal")
    plt.xlabel("True document year")
    plt.ylabel("Predicted year")
    plt.title("Gaussian Process Regression on document year")
    plt.legend(fontsize=8)
    plt.tight_layout()
    plt.savefig(path, dpi=200)
    plt.close()


def plot_wordcloud(corpus, theta, lda, vec, path):
    vocab = vec.get_feature_names_out()
    n_topics = lda.components_.shape[0]
    cols = 3
    rows = int(np.ceil(n_topics / cols))
    fig, axes = plt.subplots(rows, cols, figsize=(8.5, 2.5 * rows))
    axes = np.array(axes).reshape(-1)
    for i, comp in enumerate(lda.components_):
        freqs = {vocab[j]: float(comp[j]) for j in comp.argsort()[::-1][:40]}
        wc = WordCloud(width=420, height=240, background_color="white",
                       colormap="viridis").generate_from_frequencies(freqs)
        axes[i].imshow(wc, interpolation="bilinear")
```

03_analyze.py (part 10)

```python
        axes[i].set_title(f"Topic {i + 1}", fontsize=10)
        axes[i].axis("off")
    for j in range(n_topics, len(axes)):
        axes[j].axis("off")
    plt.tight_layout()
    plt.savefig(path, dpi=180)
    plt.close()


# ---------------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------------
def main() -> int:
    print("[1] Loading corpus...")
    corpus = load_corpus()
    corpus.to_csv(PROC / "corpus_chunks.csv", index=False)
    print(f"    {len(corpus)} chunks "
          f"({(corpus.category == 'plan').sum()} plan, "
          f"{(corpus.category == 'speech').sum()} speech) "
          f"from {corpus.source_file.nunique()} source documents")

    print("[2] Selecting number of topics (perplexity)...")
    best_k, perp_rows = select_k(corpus, candidates=(4, 6, 8, 10, 12))
    print(f"    perplexity by K: {[(r['k'], round(r['perplexity'], 1)) for r in perp_rows]}")
    # In practice perplexity keeps falling; pick a moderate K for interpretability.
    chosen_k = 8
    print(f"    chosen K = {chosen_k} (balance of perplexity and interpretability)")

    print(f"[3] Fitting final LDA with K={chosen_k}...")
    lda, vec, theta = fit_lda(corpus, chosen_k)
    topics = topic_top_words(lda, vec, n=15)
    for i, ws in enumerate(topics):
        print(f"    Topic {i + 1}: {', '.join(ws[:10])}")

    np.save(PROC / "theta.npy", theta)
    pd.DataFrame(theta, columns=[f"T{i+1}" for i in range(chosen_k)]).to_csv(
        PROC / "doc_topic.csv", index=False
    )
    (PROC / "topics.json").write_text(json.dumps(topics, indent=2))

    print("[4] GP classification (plan vs speech) on topic features...")
    y_cat = (corpus["category"] == "speech").astype(int).values
    gpc = gp_classify(theta, y_cat)
    base = baseline_classify(theta, y_cat)
```

03_analyze.py (part 11)

```python
    print(f"    GP   acc = {gpc['cv_acc_mean']:.3f} ± {gpc['cv_acc_std']:.3f}, "
          f"F1 = {gpc['cv_f1_mean']:.3f}")
    print(f"    LR   acc = {base['cv_acc_mean']:.3f} ± {base['cv_acc_std']:.3f}, "
          f"F1 = {base['cv_f1_mean']:.3f}")

    print("[5] GP regression on document year...")
    years = corpus["year"].values.astype(float)
    gpr = gp_regress_year(theta, years)
    base_reg = baseline_regress_year(theta, years)
    print(f"    GP   MAE = {gpr['cv_mae_mean']:.2f} yr, R^2 = {gpr['cv_r2_mean']:.3f}")
    print(f"    Ridge MAE= {base_reg['cv_mae_mean']:.2f} yr, R^2 = {base_reg['cv_r2_mean']:.3f}")

    print("[5b] Thresholding continuous output (year) -> era classes; GP classification...")
    era = np.array([year_to_era(int(y)) for y in years])
    n_pre, n_post = int((era == 0).sum()), int((era == 1).sum())
    gpc_era = gp_classify(theta, era)
    base_era = baseline_classify(theta, era)
    print(f"    threshold = {FEDERAL_THRESHOLD_YEAR}; pre-federal={n_pre}, federal={n_post}")
    print(f"    GP   acc = {gpc_era['cv_acc_mean']:.3f} ± {gpc_era['cv_acc_std']:.3f}, "
          f"F1 = {gpc_era['cv_f1_mean']:.3f}")
    print(f"    LR   acc = {base_era['cv_acc_mean']:.3f} ± {base_era['cv_acc_std']:.3f}, "
          f"F1 = {base_era['cv_f1_mean']:.3f}")

    metrics = {
        "perplexity": perp_rows,
        "chosen_k": chosen_k,
        "gp_classifier": {k: v for k, v in gpc.items() if k != "model"},
        "baseline_classifier_lr": base,
        "gp_regressor_year": {k: v for k, v in gpr.items() if k not in ("pred_mean", "pred_std")},
        "baseline_regressor_ridge": base_reg,
        "era_classification": {
            "threshold_year": FEDERAL_THRESHOLD_YEAR,
            "class_labels": ["pre-federal (<=2015)", "federal (>2015)"],
            "n_pre": n_pre,
            "n_post": n_post,
            "gp": {k: v for k, v in gpc_era.items() if k != "model"},
            "baseline_lr": base_era,
        },
        "topic_top_words": topics,
        "corpus_summary": {
            "n_chunks": int(len(corpus)),
            "n_plan_chunks": int((corpus.category == "plan").sum()),
            "n_speech_chunks": int((corpus.category == "speech").sum()),
            "n_source_docs": int(corpus.source_file.nunique()),
```

03_analyze.py (part 12)

```python
            "n_plan_docs": int(corpus.loc[corpus.category == "plan", "source_file"].nunique()),
            "n_speech_docs": int(corpus.loc[corpus.category == "speech", "source_file"].nunique()),
            "vocab_size": int(len(vec.get_feature_names_out())),
            "total_words": int(corpus["text"].str.split().str.len().sum()),
            "year_min": int(years.min()),
            "year_max": int(years.max()),
        },
    }
    (PROC / "metrics.json").write_text(json.dumps(metrics, indent=2))

    print("[6] Generating figures...")
    plot_perplexity(perp_rows, FIG / "fig_perplexity.png")
    plot_topic_words(topics, FIG / "fig_topic_words.png")
    plot_topic_by_category(corpus, theta, FIG / "fig_topic_category.png")
    plot_topics_over_time(corpus, theta, FIG / "fig_topics_over_time.png")
    plot_tsne(theta, y_cat, FIG / "fig_tsne.png")
    plot_confusion(np.array(gpc["confusion"]), FIG / "fig_confusion.png",
                   labels=("Plan", "Speech"), title="GP classifier: genre")
    plot_confusion(np.array(gpc_era["confusion"]), FIG / "fig_confusion_era.png",
                   labels=("Pre-2015", "Federal"), title="GP classifier: era (thresholded year)")
    plot_gpr(years, gpr["pred_mean"], gpr["pred_std"], FIG / "fig_gpr_year.png")
    plot_wordcloud(corpus, theta, lda, vec, FIG / "fig_wordclouds.png")
    print(f"    figures written to {FIG}")

    print("\nDone.")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```